

PXRD Fetcher — how the extraction pipeline works

A visual walkthrough of the four core modules, written for adapting the approach to other figure-digitization problems (e.g. sorption isotherms).

Aaron Gao · PEAK-COF project · July 2026 · files: v2/vision.py · v2/calibrate.py · v2/extract.py · tools/quality_filter.py

The core idea: three specialists, not one model

LLM — SEMANTICS

Reads what only a human could: *where* the PXRD panels are, what the axes mean, how many curves overlap, which legend label belongs to which color — and drops a **coarse anchor polyline** on each curve.

CLASSICAL CV — PRECISION

Deterministic pixel work: finds the plot frame, tick marks and curve ink, then **snaps the LLM's rough guess onto real pixels**. Reproducible, fast, no hallucination possible.

VERIFICATION — HONESTY

Every trace is scored **against the original image pixels** (never against its own output — no circularity). Fails the gates → the curve is rejected, not shipped.

Pipeline at a glance

1

Locate panels

LLM scans PDF pages, returns bounding boxes of PXRD panels + captions.

2

Adaptive re-render

Panel re-rendered from vector PDF at the DPI that makes it ~1500 px wide (clamped 220–900 DPI). Small inset → high DPI.

3

Deep read

LLM reads axis titles, tick values, series list (color, label, material) and drops anchor points along every curve.

4

Reframe check

If curve ink hugs the crop edge, the crop was off — shift & re-render once (self-correcting).

5

Frame & ticks

Evidence-scored plot-frame detection (VERIFIED vs OPEN edges) + x-tick pixel centers on a regular grid.

6

Calibration consensus

Three independent pixel→2θ fits must agree within 0.5° before any is trusted.

7

Snap & verify

Anchor polyline snapped to ink column-by-column; precision / recall / snap-rate gates decide accept or reject.

8

Postprocess & QC

Baseline (ALS), normalize, smooth; offline two-layer quality filter audits every accepted curve.

LLM stage deterministic CV verification gate

Scale: 2,370 papers → 4,664 figures → 11,445 curves (7,713 clean)

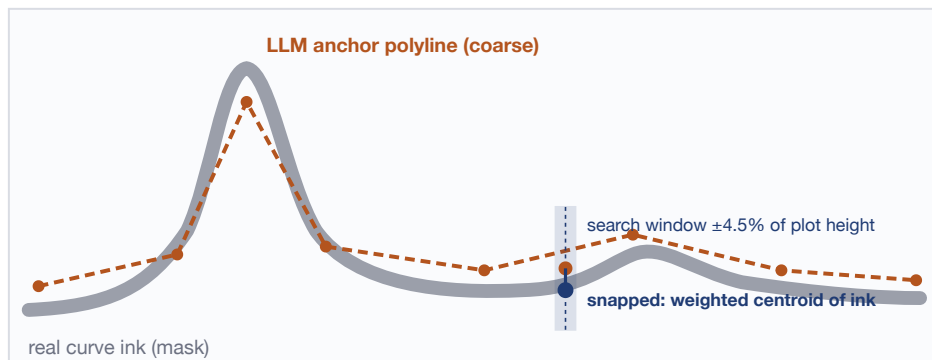
Who does what

FILE		OWNS	KEY EXPORTS
v2/vision.py	CV	All pixel primitives	detect_plot_frame · finalize_frame · detect_x_tick_pixels · color_mask_for · snap_trace · trace_fidelity
v2/calibrate.py	verify	Pixels → physical units	fit_from_pairs · pairing_candidates · fits_agree · consensus_calibration
v2/extract.py	CV verify	Per-curve orchestration	extract_series (repair ladder → gates → confidence → calibrated curve)
tools/quality_filter.py	verify	Offline corpus QC	series_verdict (waveform rules) · evidence_reasons (trace evidence)

Recommended reading order: this document → docs/pxrd_pipeline_slides.html (interactive, stage by stage) → vision.py → calibrate.py → extract.py → quality_filter.py. The shared bundle contains no data, no API keys, no .env.

vision.py The snap: from rough LLM guess to pixel truth

The LLM is good at "the red curve goes roughly here" and bad at "y = 312.4 px". So the LLM only provides a coarse polyline; `snap_trace()` walks every pixel column and pulls that guess onto the actual ink:



Per pixel column x:

- 1 interpolate anchor $\rightarrow$ expected y
- 2 collect ink pixels within the window
- 3 snap to distance-weighted centroid
$$w = 1 / (1 + (d / (\text{win}/2))^2)$$
- 4 suppress isolated jumps
5-pt median $\cdot \max(6\text{px}, 5\% h)$
- 5 trim unsnapped head/tail runs —
never fabricate data where the figure has no ink

The centroid weighting resists grabbing a neighboring curve: ink near the anchor dominates, far ink is discounted quadratically. This is what makes overlapping multi-curve figures tractable — and it transfers unchanged to hysteresis loops.

ALSO IN VISION.PY — FRAME DETECTION

`detect_plot_frame` + `finalize_frame` score long dark pixel runs as axis candidates. Each edge is tagged **VERIFIED** (a real drawn line) or **OPEN** (no line there — publication styles with only left+bottom spines). Downstream code trusts VERIFIED edges more. Ticks: `detect_x_tick_pixels` finds short stubs under the bottom edge, then `_regularize_tick_grid` keeps only centers on an arithmetic grid (snap tolerance 0.27 of the spacing).

ALSO IN VISION.PY — THE HONESTY METRIC

`trace_fidelity` compares the final trace against the **original ink mask**:

precision — of the columns I traced, how many sit on real ink?

recall — of the columns where ink is *expected*, how many did I explain?

snap rate — how often did the snap actually find pixels?

Scoring against the source image (not the model's own output) is what keeps the check non-circular — but the *reference frame* matters as much as the pixels. See the case study below.

△ CASE STUDY — A CIRCULARITY TRAP WORTH LEARNING FROM

Recall is supposed to answer "did I explain the ink that is there?" The original implementation measured it inside a band around **the finished trace**, and skipped columns the trace had abandoned ($y = \text{NaN}$). Both choices are circular: ink the trace never went near could not enter the denominator. Two consequences, neither visible without testing the metric itself:

1. Since $\text{tol} < \text{band}$, $\text{recall} \geq \text{precision}$ became an algebraic identity — verified across all **11,445 curves: zero exceptions**. The $\text{recall} \geq 0.60$ gate was dead code that never once rejected a curve.

2. Worse, it inverted the incentive: **the more of a curve a trace abandoned, the higher it scored**.

Fix: anchor the denominator on the LLM polyline (which spans the whole curve) and let abandoned columns count against you. Legitimate trimming stays free — where the anchor sits on blank canvas there is no ink to explain.

TRACE COVERING...	RECALL BEFORE	RECALL AFTER
100% (correct)	0.991	0.991
50% of the curve	0.983	0.491
25% of the curve	0.965	0.236
10% of the curve	1.000	0.092

A trace that threw away 90% of the data scored a **perfect 1.0**. After the fix, recall tracks coverage.

The transferable lesson: a verification metric is code too, and it needs its own adversarial test — feed it a deliberately fabricated and a deliberately truncated trace and confirm it screams. Ours had 10 tests; none did this.

Continued overleaf: the pixel primitives underneath the snap — frame topology, tick regularization, ink separation, color masks.

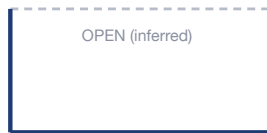
The pixel primitives underneath

Everything above depends on three deterministic measurements made before any curve is traced: *where is the plot box*, *where are the ticks*, and *which pixels are curve ink*. Each is scored on evidence rather than assumed.

FRAME TOPOLOGY — VERIFIED VS OPEN EDGES



full box: 4 × VERIFIED

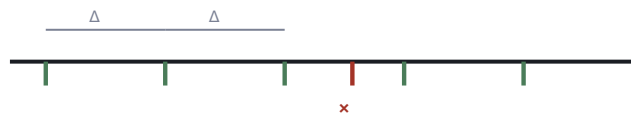


L-spines: 2 VERIFIED + 2 OPEN

Long dark pixel runs are scored as edge candidates; each edge is tagged by whether a real drawn line supports it. Spine-only publication styles are recovered by

`_frame_from_bottom_line`: find the x-axis line, infer the rest from ink extent. Downstream calibration weights VERIFIED edges more.

TICK REGULARIZATION — ONLY ARITHMETIC GRIDS SURVIVE



legend stub / stray mark rejected: $>0.27 \cdot \Delta$ off the grid

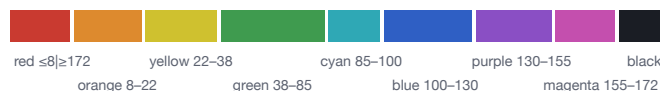
`_regularize_tick_grid` keeps only tick centers that sit within 27% of one spacing of a common arithmetic grid — stray marks, minor ticks and legend stubs fall out before any value is paired.

INK SEPARATION (BUILD_CURVE_FOREGROUND)

```
bg = P75(gray) · ink = gray < max(80, bg-45) · fg = ink
u (S>60 & gray<235) — text boxes
```

Background level is estimated from the image itself (75th percentile), so faded scans and gray-tinted panels still separate. OCR text boxes are punched out of the mask. **Frame strokes are deliberately kept**: flat PXRD baselines hug — often overlap — the bottom axis, and deleting border bands would delete real data. The anchor-centered snap weighting is what keeps axis lines from hijacking a trace instead.

COLOR VOCABULARY (COLOR_MASK_FOR)



The LLM names each curve's color in plain English; the mask is a fixed OpenCV-HSV hue window (chroma $S>60$, $V>40$), plus luminance rules for black / gray / brown. `color_mask.n` foreground is rung 1 of the repair ladder — the strictest, cleanest pixel set for multi-curve figures.

PERSIST THE MODEL'S RAW GEOMETRY FROM DAY ONE

For most of this project, `result.json` stored each curve's metadata, evidence and final data — but **not the LLM anchor polyline or the chosen frame**. Calibration could be replayed offline; tracing could not, so re-scoring a curve meant re-asking the model (an API call), and offline audits that re-derived the frame sometimes got a different one, misaligning sharp peaks by a few pixels.

Both are persisted now (`SeriesResult.anchor_points`, `FigureResult.frame_bbox`) — they cost a few hundred bytes and make every downstream metric re-derivable for free. **Build this in from day one**; retrofitting it after a corpus run means the old corpus stays unplayable.

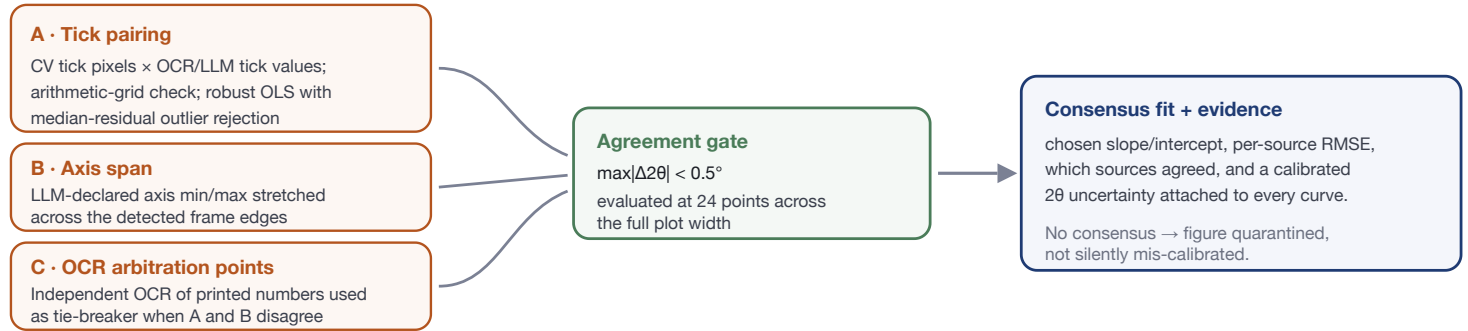
WHY THESE PRIMITIVES, IN THIS ORDER

Each stage narrows what the next can get wrong:

frame bounds where data can exist → **ticks** only mean something inside a known frame → **foreground** excludes text/legend so the snap cannot latch onto a label → **color mask** narrows to one series so overlapping curves stay separable.

Get the frame wrong and every later stage inherits the error silently — which is why frame edges carry VERIFIED/OPEN evidence rather than being assumed, and why an unsupported frame is grounds to quarantine the figure rather than proceed with a guess.

Converting pixels to 2θ is where silent catastrophes live — one misread tick label shifts every peak. So no single reading is ever trusted: three independent sources each produce a linear fit $2\theta = a \cdot x + b$, and fits only count when they **agree within 0.5° everywhere across the plot width**.



ROBUST FIT (FIT_FROM_PAIRS)

Up to two rounds: fit OLS → if the worst point's residual exceeds $\max(0.3^\circ, 3 \times \text{median_residual})$, drop it and refit. The median—not the RMSE—sets the scale, because one corrupted OCR digit drags OLS so far it would mask itself.

WORKED EXAMPLE

Four ticks detected at pixels 100 / 300 / 500 / 700; OCR reads 5 / 15 / 25 / 35:

$$\text{slope} = 10^\circ / 200 \text{ px} = 0.05^\circ/\text{px} \rightarrow 2\theta(x) = 0.05 \cdot x + 0^\circ$$
$$\rightarrow 2\theta(412 \text{ px}) = 20.6^\circ$$

Every downstream data point is just this line evaluated at a column index — which is why one wrong pairing corrupts *everything*, and why the consensus gate exists.

WHY THE Y-AXIS IS DELIBERATELY NOT CALIBRATED

Powder intensities are published in arbitrary units, stacked with arbitrary offsets, and scaled per journal style — a y-axis number carries no physical meaning across figures. The pipeline therefore records **relative intensity only** (per-curve normalized after baseline removal).

within-curve shape: trusted

cross-curve absolute intensity: never claimed

PHYSICS PLAUSIBILITY WINDOWS

A fit is discarded unless the calibrated axis lands in a physically sane window, evaluated over the *data extent* (frame edges can overshoot on multi-panel figures):

$-5^\circ \leq 2\theta_{lo}$

$2\theta_{hi} \leq 130^\circ$

$1.5^\circ \leq \text{span} \leq 135^\circ$

△ These constants are the PXRD-specific prior — the first thing to swap for a different figure type (isotherms: $P/P_0 \in [0,1]$, uptake ≥ 0 , log-axis option).

WHEN COUNTS DON'T MATCH (PAIRING_CANDIDATES)

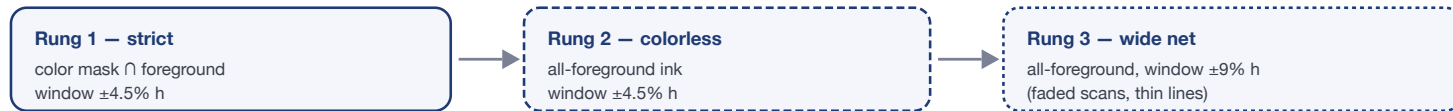
OCR often reads 4 values under 5 ticks (one label clipped or unreadable). Rather than guess, **every plausible alignment** of values onto ticks becomes a candidate pairing; a candidate survives only if its values form an arithmetic sequence (relative tolerance 8%) and its fit passes the physics window. The agreement gate then picks among survivors — alignment errors produce wildly different slopes, so wrong candidates almost never find a second witness.

FAILURE → RESPONSE LADDER

SITUATION	RESPONSE
One OCR digit corrupted	robust fit drops it (median-residual rule)
Tick fit vs axis-span fit disagree $>0.5^\circ$	OCR arbitration points break the tie
Still no two sources agreeing	figure quarantined with reasons — never shipped mis-calibrated

extract.py **Per-curve orchestration: the repair ladder and the gates**

For each curve, `extract_series()` tries increasingly permissive pixel masks until the trace passes the gates — precise first, forgiving only when needed:



Each rung is scored $0.4 \cdot \text{snap} + 0.4 \cdot \text{precision} + 0.2 \cdot \text{recall}$ — best attempt kept; ladder exits early once all gates pass.

The rung used is recorded as evidence (`repair_rounds`), so downstream QC knows how hard the curve fought.

ACCEPTANCE GATES (ALL MUST PASS)

snap_rate ≥ 0.55

precision ≥ 0.70

recall ≥ 0.60

confidence ≥ 0.55

confidence = $0.30 \cdot \text{snap} + 0.30 \cdot \text{precision} + 0.25 \cdot \text{recall} + 0.15 \cdot \text{anchor_agreement}$

anchor_agreement penalizes traces that drifted far from the LLM's polyline — if CV and LLM tell different stories, confidence drops and the curve is more likely to be rejected than silently wrong.

AFTER ACCEPTANCE

Pixels → calibrated 2θ via the consensus fit; y flipped (pixel y grows downward); the trace is **clipped to the calibrated tick span** — segments outside it belong to neighboring panels or hallucinated extensions. If $<30\%$ of the trace survives the clip, the whole curve is rejected. Then: resample at 0.02° , Savitzky-Golay(11, 3) smoothing, per-curve normalization.

THE EVIDENCE RECORD EVERY ACCEPTED CURVE CARRIES

snap_rate	fraction of columns where ink was actually found
mean_snap_residual_px	average pull distance from anchor to ink
pixel_precision / recall	fidelity vs the original ink mask
anchor_vs_final_rmsd_frac	how far the trace drifted from the LLM's guess
repair_rounds	which rung of the ladder finally worked
notes ("mask=color:red")	which pixel mask produced the accepted trace

CONFIDENCE, WORKED

A clean single-curve figure:

$0.30 \cdot 0.92 + 0.30 \cdot 0.88 + 0.25 \cdot 0.75 + 0.15 \cdot 0.85 = 0.86 \checkmark$

A curve that only half-snapped:

snap 0.58 $\checkmark$ · prec 0.71 $\checkmark$ · recall 0.45 $\times \rightarrow$ rejected

Gates are AND-ed: a good composite score cannot buy back a failed individual gate.

REJECTION TAXONOMY — WHAT EACH REASON USUALLY MEANS

REJECT_REASON	TYPICAL CAUSE
anchor polyline too short	LLM saw a legend swatch or axis label as a "curve"
no usable pixel mask	declared color absent from panel (misabeled legend)
fidelity below threshold (...)	heavy curve overlap, dashed/dotted lines, watermark scans
only N% inside calibrated window	trace ran into a neighboring panel or hallucinated extension

POSTPROCESS CHOICES (WHY THESE, NOT OTHERS)

ALS baseline ($\lambda=10^7$, $p=0.01$): the asymmetry $p \rightarrow 0$ makes the fitted baseline hug the *underside* of the signal — peaks are preserved, broad background leaves. Stiff λ keeps genuine amorphous humps in the data rather than in the baseline.

Savitzky-Golay(11, 3): fits local cubics, so peak heights and widths survive far better than under a moving average — critical when FWHM is a scientific output, not a nuisance.

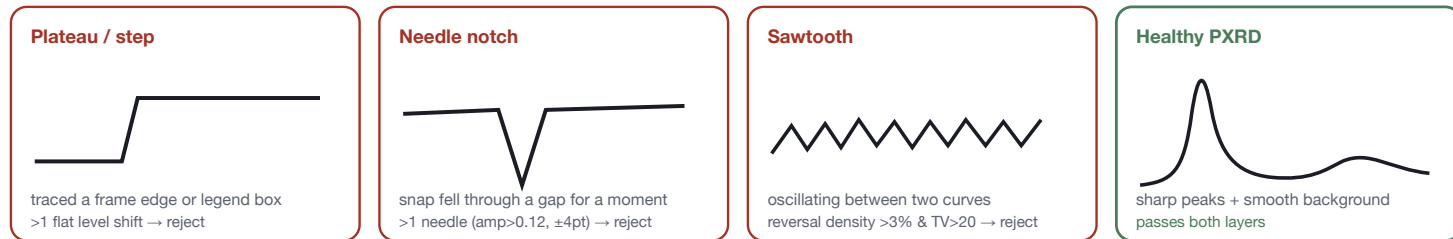
0.02° resample: uniform grid, comparable across all 11,445 curves.

DESIGN RULE WORTH STEALING

Every rejection carries a machine-readable *reason* and every acceptance carries *evidence* (snap rate, residuals, mask used, repair rounds, anchor deviation). Nothing is a silent boolean. This is what made the offline quality filter — and every audit since — possible without re-running the pipeline.

quality_filter.py Offline QC: catching what real-time gates miss

Real-time gates judge *tracing* quality. But some failures produce beautifully-traced garbage — the tracer faithfully followed the wrong thing (an axis, a grid line, a neighboring curve's tail). Those show up as **waveform pathologies** in the exported CSV, so a second, purely offline layer re-reads every accepted curve:



LAYER 1 — WAVEFORM SHAPE RULES	LAYER 2 — TRACING EVIDENCE RULES	MEASURED RESULTS
Rolling-std segmentation (window 15 pts $\approx 0.30^\circ$; "flat" < 0.004) finds plateaus ≥ 20 pts; level shifts > 0.10 with transitions < 60 pts count as steps. Plus needle-notch and sawtooth detectors shown above.	Re-reads the evidence saved at extraction time with stricter offline thresholds : precision ≥ 0.80 , snap rate ≥ 0.80 , mean residual ≤ 15 px. Curves that passed the permissive real-time gates but sit in the gray zone get filtered here.	Calibrated on a 250-figure human-audited sample: clean-set accuracy 94.4% → 98.5% . Caught 11/11 known-bad figures held out-of-sample (the KNOWN_BAD list in the file).

HOW THE THRESHOLDS WERE SET (NOT HAND-WAVED)

A random 250-figure sample was human-verdicted (good / bad trace). Thresholds were then chosen on that audit to maximize bad-curve capture at near-zero cost to good curves, and validated against 11 independently known-bad figures held out-of-sample — all 11 caught. Because the filter is offline (it re-reads CSVs + stored evidence, never re-traces), its rules can keep evolving as new failure modes appear, at zero pipeline cost.

Constants reference (the tuning surface)

CONSTANT	VALUE	MEANING
window_frac (snap)	0.045 → 0.09	vertical search half-window as fraction of plot height (rung 1 → rung 3)
jump suppression	max(6 px, 5% h)	single-column deviations larger than this are median-repaired
_AGREEMENT_TOL_DEG	0.5°	max $ \Delta 2\theta $ for two calibration fits to count as agreeing
2 θ plausibility	$[-5^\circ, 130^\circ]$, span 1.5–135°	PXRD-specific physics prior on any calibrated axis
gates (real-time)	0.55 / 0.70 / 0.60 / 0.55	snap / precision / recall / confidence minimums. The recall gate only began binding once the metric was fixed (p.2) — thresholds calibrated against the old, inflated recall are worth re-deriving.
gates (offline)	0.80 / 0.80 / 15 px	precision / snap / mean residual — stricter second pass
panel re-render	1500 px, 220–900 DPI	adaptive DPI so every panel is analyzed at comparable resolution
postprocess	ALS $\lambda=10^7$, $p=0.01$ · SavGol(11,3) · 0.02°	baseline, smoothing, resample step

Adapting this to sorption isotherms — what transfers, what breaks

The architecture (LLM semantics → CV snap → non-circular verification → offline QC) is curve-type-agnostic. Four specific things deserve attention:

⚠ 1 · DISCRETE SYMBOLS, NOT A CONTINUOUS LINE

Isotherms are plotted as markers (circles/squares), often with gaps. `snap_trace` assumes ink in (nearly) every column — `snap_rate` will crater on marker plots. **Change:** detect marker centroids (blob detection on the color mask) and fit the polyline through them, instead of per-column snapping. The anchor-weighting idea survives; the column loop becomes a blob loop.

⚠ 2 · LOG-SCALE PRESSURE AXES

`fit_from_pairs` is strictly linear (`polyfit deg 1`) and `_is_arithmetic` validates tick values as an arithmetic grid. Low-pressure isotherms use $\log(P/P_0)$ axes. **Change:** add a log-space fit candidate (fit $\log_2$ vs $\log_{10}$ (value)) and a geometric-grid tick validator; let the consensus mechanism pick linear vs log.

⚠ 3 · PHYSICS PRIORS ARE PXRD-SPECIFIC

The plausibility windows ($2\theta \in [-5^\circ, 130^\circ]$...) and the waveform rules (sharp peaks + smooth background) encode powder-diffraction physics. Isotherm equivalents: $P/P_0 \in [0, 1.05]$, uptake ≥ 0 and (per branch) **monotonically non-decreasing** — which is actually a *stronger* QC rule than anything PXRD allows. A step in an isotherm is real physics (gate opening), not an artifact: the plateau/step detector must be retired or inverted.

✓ 4 · HYSTERESIS LOOPS COME FOR FREE

Adsorption/desorption branches overlap on the same x-range — two y values per column. This looks scary but is exactly the multi-curve problem the anchor-weighted snap already solves: the LLM drops one anchor polyline per branch, and the distance weighting keeps each snap on its own branch. No architectural change needed.

Suggested porting order

#	STEP	WHY FIRST
1	Run the pipeline as-is on 5 isotherm figures	Baseline: see exactly which gates fail and how (the evidence fields tell you)
2	Swap physics priors in <code>calibrate.py</code>	Small, isolated constants; unblocks calibration consensus
3	Marker-centroid mode in <code>vision.py</code>	The one real algorithm change; everything downstream is unchanged
4	Log-axis fit candidate in <code>calibrate.py</code>	Needed for micropore region; consensus machinery already arbitrates
5	Rewrite waveform QC rules in <code>quality_filter.py</code>	Last — needs a corpus of traced isotherms to calibrate thresholds against

ONE-LINE SUMMARY

Let the LLM read the figure like a chemist, let classical CV measure it like an instrument, and never ship a curve that can't prove — against the original pixels — that it earned its numbers.